PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ

FACULTAD DE CIENCIAS E INGENIERÍA

ALGORITMIA Y ESTRUCTURA DE DATOS

Primer Examen
(Segundo Semestre 2025)

Duración: 2h 50 min.

- **No puede utilizar apuntes, solo hojas sueltas en blanco.**
- En cada función el alumno deberá incluir, a modo de comentario, la forma de solución que utiliza para resolver el problema. De no incluirse dicho comentario, el alumno perderá el derecho a reclamo en esa pregunta.
- No puede emplear plantillas o funciones no vistas en los cursos de programación de la especialidad.
- Los programas deben ser desarrollados en el lenguaje C++. Si la implementación es diferente a la estrategia indicada o no la incluye, la pregunta no será corregida.
- Un programa que no muestre resultados coherentes y/o útiles será corregido sobre el 50% del puntaje asignado a dicha pregunta.
- Debe utilizar comentarios para explicar la lógica seguida en el programa elaborado. El orden será parte de la evaluación.
- **Solo está permitido acceder a la plataforma de PAIDEIA, cualquier tipo de navegación, búsqueda o uso de herramientas de comunicación se considera plagio por tal motivo se anulará la evaluación y se procederá con las medidas disciplinarias dispuestas por la FCI.**
- Para esta evaluación solo se permite el uso de las librerías **iostream, iomanip, climits cstring, cmath o fstream**
- Su trabajo deberá ser subido a PAIDEIA.
- **Es obligatorio usar como compilador CLion.**
- Los archivos deben llevar como nombre su código de la siguiente forma codigo_EX1_P#& (donde # representa el número de la pregunta a resolver y & representa la alternativa a o b)

Elija las preguntas para obtener un puntaje de 20, si escoge más de esa nota se considera la respuesta con más bajo puntaje:

Pregunta 1 (5 puntos)

Como analista de seguridad en un SOC (*Security Operations Center*) usted recibe cada día una lista de alertas de incidentes de seguridad que debe atender. Debe planificar qué alertas atender durante su turno de 08:00 a 17:00, sin trabajar en la franja de 13:00–14:00 (hora de almuerzo). Cada alerta tiene: prioridad (1, 2 o 3), hora de inicio (0–23) y duración (horas). Las alertas de prioridad 1 requieren 1 hora adicional inmediata para el reporte post-incidente. Usted debe seleccionar una combinación de alertas sin traslapes que cumpla con una determinada cantidad de alertas por prioridad que le indique el Oficial de Seguridad de la Información.

Se le pide que, usando la estrategia de ***fuerza bruta***, elabore un programa en C++ para que cumpla con el SLA que le indique el Oficial de Seguridad.

Considera esta lista de alertas de incidentes de seguridad:

Código Alerta	Prioridad	Hora inicio	Duración (horas)
1801	2	8	1
1802	1	8	1
1803	1	9	1
1804	3	9	3
1805	1	10	1
1806	1	11	1
1807	3	12	1
1808	1	14	1
1809	2	14	2
1810	3	14	1
1811	2	15	1
1812	3	16	1
1813	2	16	2

```
{
   {1801, 2,  8, 1},
   {1802, 1,  8, 1},
   {1803, 1,  9, 1},
   {1804, 3,  9, 3},
   {1805, 1, 10, 1},
   {1806, 1, 11, 1},
   {1807, 3, 12, 1},
   {1808, 1, 14, 1},
   {1809, 2, 14, 2},
   {1810, 3, 14, 1},
   {1811, 2, 15, 1},
   {1812, 3, 16, 1},
   {1813, 2, 16, 2}
};
```

Caso de prueba 1: **P1**=3, **P2**=1, **P3**=1
1801 P2 | 1803 P1 | 1806 P1 | 1808 P1 | 1812 P3

Caso de prueba 2: **P1**=2, **P2**=2, **P3**=2
1801 P2 | 1803 P1 | 1806 P1 | 1810 P3 | 1811 P2 | 1812 P3

Caso de prueba 3: **P1**=1, **P2**=2, **P3**=3
1801 P2 | 1803 P1 | 1807 P3 | 1810 P3 | 1811 P2 | 1812 P3
1801 P2 | 1805 P1 | 1807 P3 | 1810 P3 | 1811 P2 | 1812 P3

Caso de prueba 4: **P1**=1, **P2**=1, **P3**=3
1801 P2 | 1804 P3 | 1807 P3 | 1808 P1 | 1812 P3
1801 P2 | 1803 P1 | 1807 P3 | 1810 P3 | 1812 P3
1801 P2 | 1805 P1 | 1807 P3 | 1810 P3 | 1812 P3
1802 P1 | 1807 P3 | 1810 P3 | 1811 P2 | 1812 P3
1803 P1 | 1807 P3 | 1810 P3 | 1811 P2 | 1812 P3
1805 P1 | 1807 P3 | 1810 P3 | 1811 P2 | 1812 P3

Pregunta 2 (5 puntos)

Se tiene tres almacenes A, B y C. Traslade los n paquetes del almacén A al C siguiendo las siguientes reglas:

Los n paquetes inician en el almacén A.
Un solo movimiento a la vez.
El paquete mayor no va sobre paquete menor.
Un paquete solo puede ser **movido a un almacén adyacente a su almacén actual**.
No se permite el movimiento directo entre almacenes **opuestos** (A→C).
Los n paquetes terminan en el almacén C.

Restricción: La solución debe tener una sola función recursiva que mueva los n paquetes. No se puede emplear iteraciones de ningún tipo. Debe leer por consola la cantidad de paquetes. Debe mostrar en la salida estándar todos los movimientos de los paquetes. No se permite el uso de variables globales. El uso de un TAD o almacén adicional invalida la pregunta.

Salida para X paquetes

```
Ingrese la cantidad de paquetes: X
Mover el paquete 1 del almacen A hacia el almacen B
Mover el paquete 1 ...
.....
```

Pregunta 3 (5 puntos)

Una empresa dedicada al almacenamiento cuenta con un sistema basado en robots apiladores. La empresa **ProManu S.A.** gestiona sus **Órdenes de Producción (OP)** en una **lista simplemente enlazada**. Cada orden tiene los siguientes datos: id (int) y tipo de producción (char).

Los tipos de producción son: A, B y C. Debido a un desperfecto en el sistema que genera las ordenes de producción, se han credo las ordenes de forma aleatoria, sin embargo, la empresa produce sus órdenes de forma intercalada por tipo de producción, es decir, siempre siguiendo el orden de A, B y C y para no afectar el orden aleatorio generado, siguiendo el orden de la lista inicial. Solo en caso de que no exista una orden de un tipo, se omite y se busca el del siguiente tipo.

Por ejemplo:

Entrada: [(7,B) → (2,A) → (9,C) → (5,A) → (6,B)]

Salida: [(2,A) → (7,B) → (9,C) → (5,A) → (6,B)]

Otro ejemplo:

Entrada: [(3,C) → (9,A) →(8,C) → (4,A) → Ø]

Salida: [(9,A) → (3,C) → (4,A) → (8,C)]

Se le pide:

Implementar una función reordenaLista que reordene la lista de acuerdo con el criterio indicado. Asuma que solo existen 3 tipos de producción (A, B y C). Para esta pregunta no puede utilizar TADs adicionales ni gestionar nueva memoria. Solo puede trabajar con la lista aleatoria inicial y punteros auxiliares si necesita.

Pregunta 4 (10 puntos)

Una empresa dedicada a la jardinería adquiere una unidad automatizada ROBOTJARDIN 1.0 que tiene la capacidad de recibir como dato de entrada una matriz (no se puede modificar), internamente cuenta con una pila auxiliar y una memoria temporal (arreglo unidimensional). Esta versión solo puede emplear funciones iterativas. Se sabe que el robot recibe una matriz que representa un jardín con los sectores marcados para cortar, pero debido a su tecnología solo puede realizar cortes formando rectángulos o cuadrados, por lo tanto, el robot debe contar con un algoritmo que empleando sus limitados recursos le permita calcular el área máxima que puede cortar. A continuación, algunos ejemplos:

Si tenemos un jardín nxm, n=4 y m=4 donde los 1s son los sectores que se desean cortar:

1	**1**	**1**	0
1	**1**	**1**	0
1	0	1	1
1	1	0	0

La solución será: 6
Ya que el rectángulo máximo esta dado por 6 sectores.

Si tenemos un jardín nxm, n=4 y m=5 donde los 1s son los sectores que se desean cortar:

1	1	1	0	0
1	**1**	**1**	**1**	**1**
1	**1**	**1**	**1**	**1**
1	1	0	0	0

La solución será: 10
Ya que el rectángulo máximo esta dado por 10 sectores.

Desarrolle el algoritmo que calcule el área máxima a cortar, la complejidad de este desarrollo debe ser como máximo *O*(nxm). **Si no cumple con la complejidad, usa más de un arreglo o emplea recursión invalida su respuesta.**

Pregunta 5 (5 puntos)

Pregunta: Caso "Banco ALG"

Contexto:
El banco **ALG** quiere optimizar la atención en ventanilla. Se atiende **por prioridad** y, dentro de cada prioridad, en **orden de llegada** (FIFO). Existen 2 tipos de clientes:

- **P1 – Preferente**: adultos mayores, gestantes, personas con discapacidad.
- **P2 – Regular**: público general.

Para reducir memoria y simplificar la administración, el banco exige **una sola estructura** que contenga las **dos subcolas en segmentos contiguos**, en este orden: [P1][P2].

Se pide:
1. Implementar una cola de prioridad eficiente.
2. Mantener operaciones *O*(1) para encolar y desencolar, preservando la estabilidad FIFO en cada prioridad.

Restricciones:
La lista siempre debe cumplir:

- Segmentos **contiguos** y **ordenados**: primero todos los P1, luego P2. (Representación: [P1][P2])
- Se mantienen **1 puntero frontera**: last1: último nodo del segmento P1 (o nullptr si P1 está vacío).
- Además, la cola tiene head (frente global) y tail (fin global).
- Debe mantener todas las reglas de inserción y atención en *O*(1) (sin recorrer toda la cola)
- Debe en cada momento actualizar los punteros de su TAD para no perder integridad.

Ejemplo de eventos.txt	**Salida esperada salida.txt**
llegada 1001 regular llegada 1002 preferente atender 1 llegada 1004 regular llegada 1005 preferente imprimir atender 3 imprimir	Atendido: 1002 (P1) Estado: [P1: 1005][P2: 1001, 1004] Atendido: 1005 (P1) Atendido: 1001 (P2) Atendido: 1004 (P2) Estado: [P1:][P2:]

Llegada → Encola en P1 o P2.
Atender K → Desencola los K primeros elementos. Muestra atención.
Imprimir → Imprime el estado de ambas colas.

Al finalizar el examen, comprima la carpeta de su proyecto empleando el programa Zip que viene por defecto en el Windows, **no se aceptarán los trabajos compactados con otros programas como RAR, WinRAR, 7zip o similares**. Luego súbalo a la tarea programa en Paideia para este examen.

Profesores del curso:

Ana Roncal
Fernando Huamán
David Allasi
Erasmo Gómez
Rony Cueva

San Miguel, 18 de octubre del 2025